VISUAL BUILDER / ОСНОВНОЙ СЦЕНАРИЙ

Программы из понятных блоков.

Человек выражает логику задачи. Java создаётся автоматически, а результат проверяется настоящей JVM.

Собрать. Настроить. Попробовать.

Шесть основных действий, обычные поля и ветки «Да / Нет». Код и инженерные метрики открываются отдельно и не нужны для первого запуска.

01 / Выбрать задачу

Приветствие, скидка, доставка, список покупок или чистый лист.

02 / Собрать схему

Блоки, условия и повторы. Плюс добавляет шаг в нужное место.

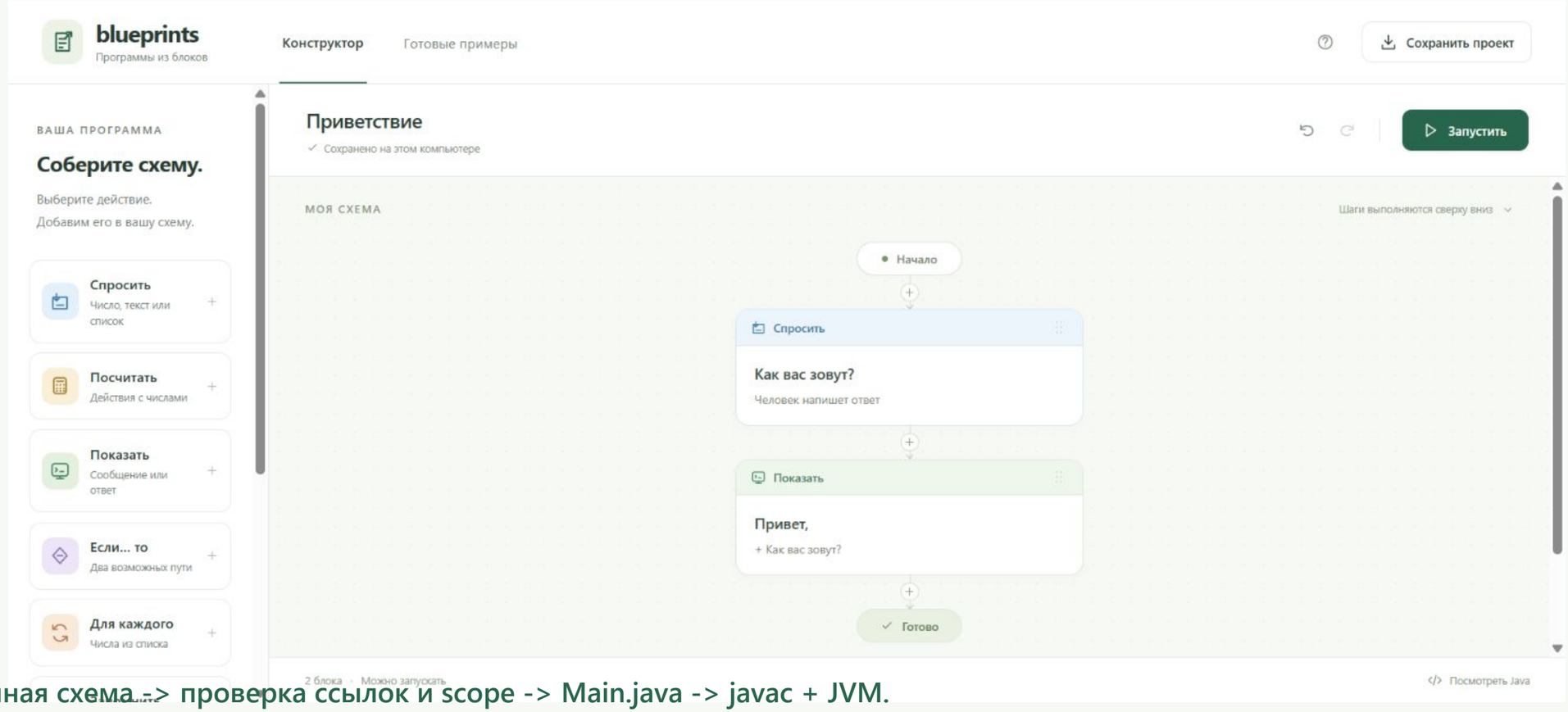
03 / Получить ответ

Понятная форма, реальная Java и пошаговое объяснение модели.

VISUAL BUILDER / ИНТЕРФЕЙС

Сначала задача. Код - по желанию.

Главная страница теперь открывает конструктор. Прежние инструменты анализа остаются в расширенной Studio.



Типизированная схема -> проверка ссылок и scope -> Main.java -> javac + JVM.

ENGINEERING / 01

Java. Видеть. Менять. Проверять.

Расширенная Studio для Java-монолита: связанная модель, объяснимый патч и проверяемый результат.

Изменение должно выдержать проверку.

Настоящий javac AST связывает код и граф. Отдельные JVM проверяют обе версии. Лаборатории помогают оценить скорость, силу тестов и причину расхождения.

17918

непустых строк продукта

205

автоматических тестов / PASS

0

runtime-библиотек и API-ключей

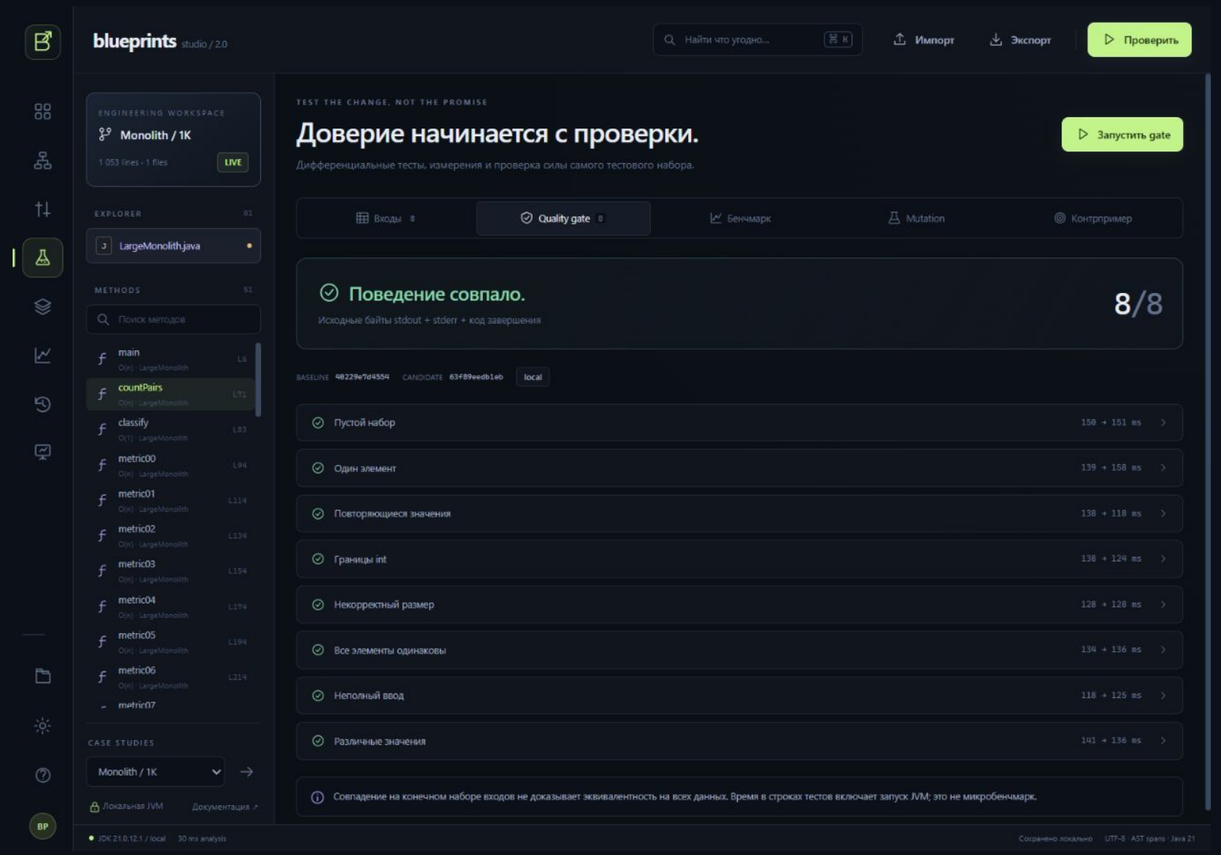
02 / WORKSPACE

Инструмент для review и живой защиты.

Скриншот локальной Studio после настоящего применения плана и запуска монолита на восьми входах.

8 связанных экранов

Source + canvas
Diff и планы
Test Lab
Архитектура
Метрики
История
Режим защиты



Проверенный результат относится к хешам baseline/candidate. После правки он становится устаревшим.

Тема, навигация, minimap, checkpoints и Ctrl+K - часть рабочего процесса.

03 / ARCHITECTURE

Один источник истины - Java-текст.

Граф не интерпретирует Java самостоятельно. Смысл программы задают стандартный компилятор и JVM.

01 / javac AST

UTF-16 source spans; методы, инструкции и контрольный поток.

02 / Analysis + plans

CFG, вызовы, данные, 25 правил, diff и source-bound предложения.

03 / Runner + evidence

Раздельные сборки, отдельные JVM, raw byte compare и manifest.

Span edit -> новый исходник -> повторный AST. Нетронутые символы и комментарии сохраняются. Планы проверяются сервером заново; stale hash и overlap отклоняются.

04 / REFACTOR PLANS

Сначала обоснование. Затем изменение.

Пять узких семейств преобразований с диапазонами, before/after, source hash и обязательным gate.

Histogram pairs

Двойной чистый обход int[] -> индекс частот. Среднее O(n), память O(k).

StringBuilder

Поддерживаемый String(String[]) accumulator -> builder; null сохраняется как текст.

Int folding / constant branch

Int overflow и shifts сохраняются; деление на ноль не сворачивается.

Guard clause

Поддерживаемый terminal return/throw убирает else, сохраняя блоки scope.

Static extraction отдельно: независимый метод -> helper + delegate с проверкой компиляции.

05 / THE OPTIMIZATION

Вложенный поиск превращается в индекс.

Чистый шаблон считает упорядоченные равные пары, включая самопары. Для частоты f получается f * f.

BEFORE / $O(n^2)$

```
int matches = 0;
for (int a : values) {
  for (int b : values) {
    if (a == b) matches++;
  }
}
return matches;
```

AFTER / O(n), EXPECTED

```
Map<Integer,Integer> f = new HashMap<>();
for (int x : values)
  f.merge(x, 1, Integer::sum);
int result = 0;
for (int n : f.values()) result += n*n;
return result;
```

Сохраняется int overflow; меняется расход памяти. OutOfMemoryError не гарантирован одинаковым. Наличие побочных эффектов отклоняет точный шаблон.

06 / TEST LAB

Проверяем изменение и сами тесты.

Длинные операции выполняются как ограниченные задачи: стадии, результат, отмена и защита от поздних ответов.

Differential gate

Две сборки и JVM. Raw stdout + stderr + exitCode. Timeout/flood всегда FAIL.

Mutation testing

AST-операторы меняются, варианты компилируются. Killed / survived / invalid.

Counterexample reduction

Бюджетный ddmin для n+n int; финальный вход повторно подтверждает расхождение.

Seeded input generation

Границы, повторы, случайные и сортированные массивы; воспроизводимый seed.

На демопроекте: 8/8 gate, 100% среди выбранных valid mutants, контрпример 6 -> 1 элемент.

07 / MEASUREMENTS

Малые данные тоже имеют значение.

Реальные локальные медианы countPairs; 3 warmup, вызов внутри JVM, clone и process startup вне таймера.

N	Baseline, мкс	Candidate, мкс	Отношение
128	48.2	71.2	0.68x
1024	305.0	132.1	2.31x
4096	4728.5	405.6	11.66x

На малом n HashMap может проиграть из-за накладных расходов. Harness не JMH: JIT, GC и среда влияют на замеры. Полные samples и environment сохранены в studio-demo.json.

08 / ANALYSIS

Структура проекта становится видимой.

У каждого статического вывода есть источник и граница применимости.

Graph intelligence

Достижимость, Tarjan SCC, доминаторы, shortest/bounded structural paths.

Project architecture

Вызовы, циклы, fan-in/out и hotspots. Лексическое разрешение, не полный dispatch.

Data flow + complexity

Чтения/записи по именам, композиционный Big O. Не SSA и не alias analysis.

Explainable diagnostics

5 базовых + 20 дополнительных правил, line/span, severity и SARIF 2.1.0.

Структурный путь не означает выполнимость условий. Exceptional flow моделируется консервативно.

09 / EVIDENCE

Review можно забрать с собой.

Автономный комплект связывает изменение, входы и результат; повторная проверка не зависит от UI.

```
evidence.zip
  baseline/*.java
  candidate/*.java
  inputs.json
  behavior-report.json
  evidence-manifest.json
  benchmark.json / mutation-report.json
  changes.patch
  report.html
```

Verified manifest выдаётся только для завершённого compare job с соответствующими hashes исходников. Произвольный клиентский отчёт не становится verified.

Дополнительно: граф SVG / Mermaid / DOT, диагностика SARIF. SHA-256 идентифицирует содержимое, но не является цифровой подписью.

10 / VERIFICATION

205 тестов. Настоящий compiler и JVM.

Одна команда python tests/verify.py компилирует, поднимает отдельный сервер, проверяет и останавливает его.

141

UNIT

Визуальные блоки, scope, Java generation, trace, diff, lexer/XSS, ZIP и storage.

22

BASE API

Java scope, UTF-16, edits, extraction, raw bytes, timeout/flood, Origin.

42

STUDIO + BUILDER API

Планы, labs, jobs, evidence и реальные запуски программ из блоков.

Локальный полный прогон PASS; удалённый CI recipe включён, но не запускался здесь. Docker не проверен. Unit diff дополнительно проверяет 300 seeded последовательностей правок.

11 / SOURCE AUDIT

Объём можно пересчитать.

UTF-8 splitlines; непустые строки включают комментарии. Объём сам по себе не является метрикой качества.

Язык	Строки	Непустые
CSS	6202	5680
HTML	561	553
Java	4781	4529
JavaScript	8785	8409
Python	1126	999

Исключены runtime, зависимости, build/dist, JSON, примеры, дубли, документация и SVG. Продукт отдельно от тестов: 17918 непустых строк.
Команда: python tests/source_audit.py.

12 / RUN & DEFEND

Готово к локальной демонстрации.

Сначала показать изменение и последствия. Затем архитектуру, тесты и честные технические границы.

```
.\start.ps1 -TrustedLocal
http://localhost:8080
```

1. Монолит -> canvas -> объяснимый diff -> apply + 8/8 gate.
2. Benchmark -> mutation score -> намеренная ошибка -> меньший контрпример.
3. Checkpoint -> extraction -> architecture -> evidence ZIP.

Границы: trusted local не sandbox; граф структурный, не полный exceptional CFG; конечные тесты не доказывают все входы. На Site - помеченные записи; произвольная Java - локально.

`http://localhost:8080 / расширенная Studio: /expert.html`